Files në Python

Alion Emini

Kurs i Programimit në Python
kurse.informatika
September 29, 2025

Contents

1 Files në Python **5**

1.1 Hyrje: pse na duhen skedarët? . . . 5

1.2 Tre hapat: *Hap — Proceso — Mbyll* . . . 6

1.3 Llojet e skedarëve: tekst dhe binar . . . 6

1.4 Metodat e aksesit: sekuenciale dhe direkte . . . 7

1.5 Emrat e skedarëve dhe objektet *file* . . . 7

1.6 Hapja e skedarit (`open()`) . . . 7

1.6.1 Specifikimi i vendndodhjes së skedarit (path) . . . 8

1.7 Shkrimi i të dhënave në skedar . . . 9

1.7.1 Mbyllja e skedarit dhe buffer-i i shkrimit . . . 9

1.7.2 Shkrimi i disa rreshtave në një skedar . . . 10

1.8 Leximi i të dhënave nga skedari . . . 11

1.8.1 Leximi i të gjithë përmbajtjes me `read()` . . . 11

1.8.2 Leximi i një rreshti me readline() . . . 12

1.8.3 Pozicioni i leximit (read position) . . . 12

1.9 Bashkimi i `\n` me një *string* . . . 13

1.10 Leximi i një *string*-u dhe heqja e `\n` . . . 14

1.11 Shtimi i të dhënave në një skedar ekzistues (`'a'`) . . . 15

1.12 Shkrimi dhe leximi i të dhënave numerike . . . 16

1.12.1 Shkrimi i numrave: nga tip numerik në *string* . . . 16

1.12.2 Leximi i numrave: nga *string* në tip numerik . . . 17

1.13 Përdorimi i cikleve për të procesuar skedarë . . . 18

1.13.1 Shkrimi i shumë vlerave me `for` (`sales.txt`) . . . 19

1.13.2 Leximi me `while` dhe zbulimi i fundit të skedarit (EOF) . . . 20

1.13.3 Leximi më i thjeshtë: `for` direkt mbi objektin e skedarit . . . 21

1.14 Përpunimi i *registreve* (records) . . . 22

1.14.1 Shkrimi i record-eve në skedar . . . 23

1.14.2 Leximi i record-eve nga skedari . . . 24

1.15 Përdorimi i `with` me skedarët (context manager) . . . 26

1.15.1 Çfarë është një context manager? . . . 26

1.15.2 Shembuj të përdorimit të `with` . . . 26

1.15.3 Çfarë ndodh jashtë bllokut `with`? . . . 28

Chapter 1

Files në Python

Objektivat e kapitullit

Në fund të këtij kapitulli, do të jesh në gjendje të:

- Shpjegosh pse duhen skedarët dhe hapat bazë *hap—proceso—mbyll*.
- Dallosh llojet e skedarëve (tekst vs. binar) dhe metodat e aksesit (sekuencial vs. direkt).
- Përdorësh `open(filename, mode)` me modalitetet `'r'`, `'w'`, `'a'` dhe të specifikosh rrugë (path) të skedarëve.
- Shkruash dhe lexosh të dhëna tekstuale; të menaxhosh `\n` dhe `rstrip()`.
- Shkruash/lexosh numra (konvertimi `str()`, `int()`, `float()`).
- Përdorësh cikle `while` / `for` për të procesuar skedarë dhe për të zbuluar EOF.
- Strukturash të dhënat si *record-e* me *field-e* dhe t'i lexosh/shkruash në mënyrë të qëndrueshme.
- Përdorësh `with` (context manager) për hapje/mbyllje automatike dhe më të sigurt të skedarëve.

1.1 Hyrje: pse na duhen skedarët?

Kur një program duhet të ruajë të dhëna për t'i përdorur më vonë, ai i *shkruan* ato në një skedar në disk. Më vonë, programi mund t'i *lexojë* të dhënat nga skedari dhe t'i rikthejë në memorie. Programet që kemi shkruar deri tani kërkojnë që përdoruesi t'i fusë të dhënat sa herë që programi ekzekutohet, sepse të dhënat që mbahen në RAM (referencuar nga variablat) *zhduken* sapo programi ndalon. Nëse duam të ruajmë të dhënat *midis* ekzekutimeve, duhet një mënyrë persistente ruajtjeje: **skedari** në disk. Pasi të dhënat të jenë shkruar në një skedar, ato *mbeten* atje edhe pasi programi përfundon dhe mund të lexohen përsëri kur të nevojiten. Shumica e paketave të zakonshme të softuerit përdorin skedarë për të ruajtur të dhëna:

- **Përpunues teksti** (p.sh. *Word*): shkruan dokumente (shkresa, raporte) dhe i ruan në skedarë për t'u redaktuar dhe printuar më vonë.
- **Redaktues imazhesh**: vizatime dhe foto (p.sh. nga kamera dixhitale) ruhen në skedarë.
- **Spreadsheet**: të dhënat numerike dhe formulat vendosen në rreshta/kolona dhe ruhen në skedarë për përdorim të mëvonshëm.
- **Lojëra**: ruajnë tabela me emra lojtarësh e pikë, madje dhe *gjendjen* e lojës për ta vazhduar më vonë.
- **Shfletues web**: ruajnë skedarë të vegjël *cookie* me informacion të sesionit (p.sh. shporta e blerjeve).

Përkufizim — Output file vs. Input file

"Shkrim" në skedar do të thotë kopjimi i të dhënave nga RAM-i (nga një variabël) *në skedar*. Skedari ku *shkruajmë* quhet **skedar daljeje** (*output file*). **"Lexim" nga skedari** do të thotë kopjimi i të dhënave nga skedari *në RAM* dhe referencimi i tyre nga një variabël. Skedari prej të cilit *lexojmë* quhet **skedar hyrjeje** (*input file*).

1.2 Tre hapat: *Hap — Proceso — Mbyll*

Sa herë që një program përdor një skedar, ndiqen gjithmonë këta tre hapa bazë:

1) **Hap skedarin.** Krijon një lidhje midis programit dhe skedarit. Hapja e një skedari *daljeje* zakonisht krijon skedarin në disk dhe lejon shkrimin. Hapja e një skedari *hyrjeje* lejon leximin.

2) **Proceso skedarin.** Shkruaj të dhëna nëse është skedar daljeje; lexo të dhëna nëse është skedar hyrjeje.

3) **Mbyll skedarin.** Kur puna përfundon, mbyllja e skedarit *shkëput* lidhjen me programin dhe siguron që të dhënat të jenë shkruar siç duhet.

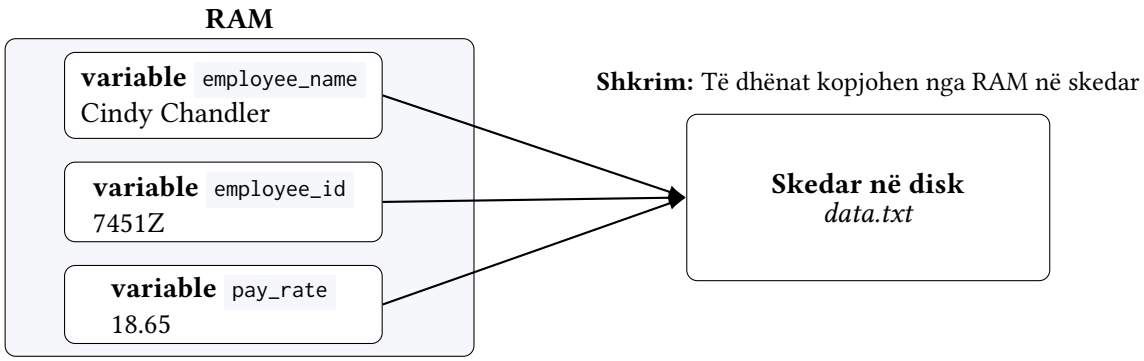

Figure 1.1: Shkrimi i të dhënave në një skedar (output).

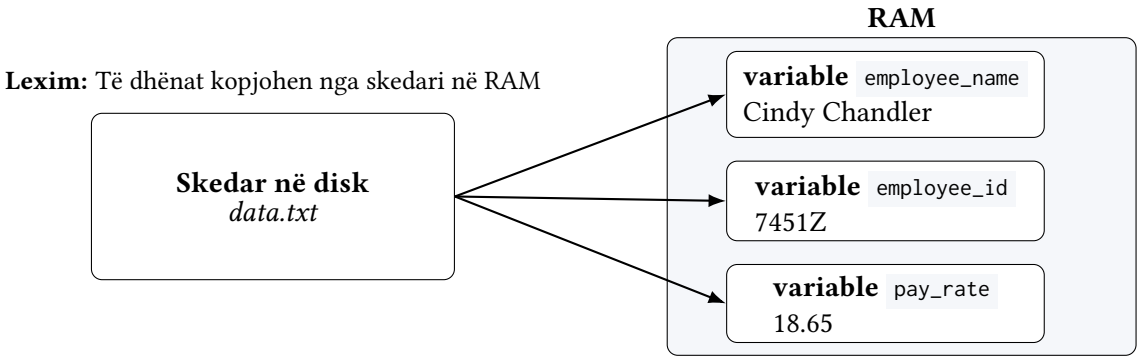

Figure 1.2: Leximi i të dhënave nga një skedar (input).

1.3 Llojet e skedarëve: tekst dhe binar

Në përgjithësi ka dy lloje kryesore skedarësh:

- **Skedarë teksti** — përmbajtja ruhet si *tekst* (ASCII/Unicode). Edhe numrat shkruhen si karaktere, ndaj mund të hapen me redaktues teksti (p.sh. *Notepad*).

- **Skedarë binarë** — përmbajtja *nuk* është konvertuar në tekst; është e destinuar për t'u lexuar nga programe. Përmbajtja nuk është e lexueshme me redaktues teksti.

Në këtë kurs do të punojmë me **skedarë teksti**, që t'i mund të inspektohen lehtësisht me një editor.

1.4 Metodat e aksesit: sekuenciale dhe direkte

Gjuhët e programimit ofrojnë dy mënyra tipike për të aksesuar të dhënat në skedar:

- **Akses *sekuencial*** — leximi bëhet nga fillimi deri në fund. Për të arritur një pjesë në fund, duhet lexuar gjithçka që vjen përpara (si një kasetë muzikore).
- **Akses *direkt* (random)** — mund të "kërceni" direkt te cilado pjesë e skedarit (si te një CD/MP3 player).

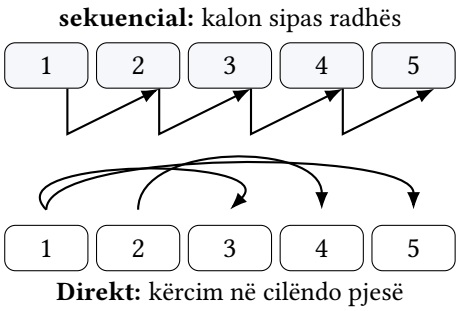

Figure 1.3: Akses *sekuencial* kundrejt aksesit *direkt*.

Në këtë kapitull do të përdorim skedarë **me akses sekuencial**, të thjeshtë për t'u trajtuar dhe mjaftueshëm për të kuptuar operacionet bazë me skedarë.

1.5 Emrat e skedarëve dhe objektet *file*

Çdo skedar identifikohet nga një **emër skedari** (*filename*). Shumë sisteme përdorin *zgjatime* (p.sh. `.jpg`, `.txt`, `.docx`), që tregojnë llojin e përmbajtjes (imazh JPEG, tekst, dokument Word, etj.). Që një program të punojë me një skedar në disk, ai krijon në memorie një **objekt skedari** (*file object*), i cili lidhet me skedarin përkatës dhe ofron metoda për lexim/shkrim. Në kod, një *variabël* referon këtë objekt dhe përdoret për të kryer operacionet mbi skedarin.

Figure 1.4: Një variabël referon një file object të lidhur me një skedar fizik në disk.

1.6 Hapja e skedarit (`open()`)

Për të punuar me një skedar, përdorim funksionin `open`. Ky funksion krijon një *file object* në memorie dhe e lidh atë me një skedar fizik në disk.

```
file_variable = open(filename, mode)
```

Në këtë format:

- `file_variable` — emri i variablës që do të referojë *file object*-in.
- `filename` — një *string* me emrin e skedarit.
- `mode` — një *string* që tregon mënyrën e hapjes (lexim, shkrim, shtim, etj.).

Modaliteti	Përshkrimi
`'r'`	Hap skedarin vetëm për **lexim**. Skedari *nuk* mund të modifikohet ose të shkruhet.
`'w'`	Hap skedarin për **shkrim**. Nëse ekziston, **fshihet përmbajtja** e tij. Nëse nuk ekziston, **krijohet**.
`'a'`	Hap skedarin për **shtim në fund**. Të gjitha të dhënat do t'i shtohen fundit të skedarit. Nëse nuk ekziston, **krijohet**.

Table 1.1: Disa nga modet më të zakonshme të skedarëve në Python

Shembull: Hapja e skedarëve për lexim dhe shkrim

```
# Hap "customers.txt" për lexim ('r')
customer_file = open('customers.txt', 'r')

# Hap ose krijon "sales.txt" për shkrim ('w')
sales_file = open('sales.txt', 'w')
```

Kujdes me `'w'`

Hapja me `'w'` *krijon* skedarin në disk dhe **e zbraz** nëse ekziston. Të dhënat e vjetra humbasin.

1.6.1 Specifikimi i vendndodhjes së skedarit (path)

Kur i kalon funksionit `open` vetëm emrin e skedarit (pa rrugë), Python supozon që skedari ndodhet në të njëjtin folder me programin që po ekzekutohet. Për shembull, nëse programi yt ndodhet te

`C:\Users\Blake\Documents\Python`

dhe ekzekuton këtë rresht:

```
test_file = open('test.txt', 'w')
```

atëherë skedari `test.txt` do të krijohet në të njëjtin folder me programin. Nëse dëshiron të hapësh ose krijosh një skedar në një vendndodhje tjetër, duhet të specifikosh edhe rrugën (path) përveç emrit të skedarit. Në Windows, kur përdor rrugë absolute me `\`, është e rëndësishme të përdorësh prefiksin `r` për të treguar që string-u është *raw string*. Kjo bën që Python të lexojë `\`-t si karaktere të zakonshme, jo si escape sequences. Për shembull:

```
test_file = open(r'C:\Users\Blake\temp\test.txt', 'w')
```

Ky rresht krijon skedarin `test.txt` në folderin `C:\Users\Blake\temp`. Nëse nuk përdor `r`, Python mund të ngatërrojë `\`-t me escape sequences dhe të japë gabim.

```python
test_file = open('test.txt', 'w')

test_file = open(r'C:\Users\Blake\temp\test.txt', 'w')
```

Këshillë praktike

Në Windows, përdor `r'...'` (raw string) për rrugë me `\` që të mos interpretohen si *escape sequences*. Në Linux/macOS, përdor `'/home/user/temp/test.txt'` ose `'/Users/blake/temp/test.txt'`.
Në projekte, prefero `pathlib.Path` për rrugë portabël (p.sh. `Path('temp') / 'test.txt'`).

1.7 Shkrimi i të dhënave në skedar

Deri tani, kemi përdorur funksione të ndryshme të bibliotekës së Python-it dhe kemi shkruar edhe funksionet tona. Tani prezantojmë një koncept të ri: **metoda**. Një *metodë* është një funksion që i përket një objekti dhe vepron mbi të. Për shembull, pasi hapim një skedar, përdorim metodat e objektit të skedarit për të kryer operacione mbi të.

Shembull — Dallimi mes funksionit dhe metodës

- **Funksion:** `len('abc')` — funksioni `len` përdoret mbi një objekt (string).
- **Metodë:** `'abc'.upper()` — metoda `upper()` thirret nga vetë objekti string.

Pasi një skedar është hapur për shkrim (`'w'` ose `'a'`), përdorim metodat e *file object*-it.

Përkufizim — Metoda `'write()'`

`file_variable.write(string)` — shkruan *string*-un në skedar. Skedari duhet të jetë hapur për shkrim (`'w'` ose `'a'`), përndryshe ndodh gabim.

```python
customer_file.write('Charles Pace')

name = 'Charles Pace'
customer_file.write(name)
```

Në shembullin e dytë, vlera e variablës `name` shkruhet në skedarin e lidhur me `customer_file`. Mund të shkruani edhe numra, por duhet t'i ktheni në string me `str()` ose `f-strings`.

1.7.1 Mbyllja e skedarit dhe buffer-i i shkrimit

Kur programi përfundon punën me një skedar, duhet ta mbyllë atë. Mbyllja e skedarit shkëput lidhjen me skedarin dhe siguron që të gjitha të dhënat e pambetura në *buffer* të shkruhen në disk. Në disa sisteme, nëse nuk mbyllet skedari, mund të humben të dhëna, sepse të dhënat fillimisht ruhen në një *buffer* në memorie për performancë më të mirë. Vetëm kur buffer-i mbushet ose skedari mbyllet, të dhënat ruhen fizikisht në disk.

```python
customer_file.close()
```

Çfarë kthen metoda `write()`?

Metoda `write()` kthen një **numër të plotë** që tregon sa karaktere u shkruan në skedar. Zakonisht ky vlerë injorohet, por mund të përdoret për të kontrolluar nëse shkrimi u krye me sukses.

```
n = customer_file.write('Charles Pace')
print(n) # -> 12 (numri i karaktereve të shkruara)
```

Programi 1 (`file_write.py`)

```
def main():
    # Hape skedarin "philosophers.txt" për shkrim.
    outfile = open('philosophers.txt', 'w')

    # Shkruaj tre emra filozofësh (secili me \n në fund).
    row1 = outfile.write('John Locke\n')
    row2 = outfile.write('David Hume\n')
    row3 = outfile.write('Edmund Burke\n')

    # Mbyll skedarin.
    outfile.close()

    # Printimi i numrit të karaktereve të shkruara në skedar.
    print(f'Numri i karaktereve të shkruara në rreshtin e parë: {row1}')
    print(f'Numri i karaktereve të shkruara në rreshtin e dytë: {row2}')
    print(f'Numri i karaktereve të shkruara në rreshtin e tretë: {row3}')


# Call the main function.
if __name__ == '__main__':
    main()
```

```
Numri i karaktereve të shkruara në rreshtin e parë: 12
Numri i karaktereve të shkruara në rreshtin e dytë: 11
Numri i karaktereve të shkruara në rreshtin e tretë: 13
```

Përmbajtja e skedarit `philosophers.txt`

```
John Locke
David Hume
Edmund Burke
```

Çdo `\n` është *newline* që ndan elementët në rreshta të veçantë kur hapet skedari në një editor teksti.

1.7.2 Shkrimi i disa rreshtave në një skedar

Në programin e mësipërm, hapet skedari `philosophers.txt` me modalitetin `'w'`. Kjo bën që skedari të krijohet (nëse nuk ekziston) dhe të hapet për shkrim. Po ashtu, krijohet një objekt skedari në memorie dhe i caktohet variablës `outfile`. Në këtë program shkruhen tre stringje në skedar. Çdo rresht përdor metodën `write()` për të shtuar një

emër filozofi, ku secili string përfundon me `\n` (escape sequence për rresht të ri). Më pas mbyllet skedari. Pas ekzekutimit të këtij programi, përmbajtja e skedarit `philosophers.txt` do të jetë si më poshtë:

```
John Locke\nDavid Hume\nEdmund Burke\n
```

Vini re se çdo string i shkruar përfundon me `\n`, që është escape sequence për rresht të ri. Kjo jo vetëm ndan elementët në skedar, por bën që secili të shfaqet në rresht të veçantë kur hapet me një editor teksti (p.sh. Notepad).

1.8 Leximi i të dhënave nga skedari

Kur një skedar është hapur për lexim (`'r'`), mund të përdorim metodat e objektit të skedarit për të marrë të dhënat nga ai. Dy metodat më të përdorura janë:

- `read()` — lexon **të gjithë** përmbajtjen e skedarit si një *string* të vetëm.
- `readline()` — lexon **një rresht** nga skedari (deri në `\n`), përfshirë edhe `\n`-në në fund të rreshtit.

1.8.1 Leximi i të gjithë përmbajtjes me `read()`

Nëse dëshiron të lexosh të gjithë përmbajtjen e një skedari në memorie, përdor metodën `read()`. Kjo është e përshtatshme kur skedari është i vogël dhe mund të mbahet i tëri në RAM.

Programi 2 (`file_read.py`) — `'read()'`

```python
def main():
    # Hap skedarin për lexim.
    infile = open('philosophers.txt', 'r')

    # Lexo të gjithë përmbajtjen si një string.
    file_contents = infile.read()

    infile.close()

    # Shtyp përmbajtjen e lexuar.
    print(file_contents)

if __name__ == '__main__':
    main()
```

```
John Locke
David Hume
Edmund Burke
```

Kur thërret `infile.read()`, përmbajtja e plotë e skedarit lexohet dhe ruhet si një string në variablën `file_contents`. Për shembull, nëse skedari përmban:

```
John Locke\nDavid Hume\nEdmund Burke\n
```

atëherë `file_contents` do të jetë një string që përmban të gjithë këtë tekst, përfshirë karakteret `\n` që ndajnë rreshtat. Kur thërret `print(file_contents)`, Python shfaq tekstin në ekran, duke respektuar karakteret e rreshtit të ri (`\n`), kështu që secili emër shfaqet në rresht të veçantë.

1.8.2 Leximi i një rreshti me `readline()`

Shpesh, programet kanë nevojë të lexojnë dhe të përpunojnë të dhënat nga skedari një nga një, p.sh. për të llogaritur totalin e disa vlerave ose për të analizuar çdo rresht veçmas. Për këtë përdoret metoda `readline()`, e cila lexon një rresht të vetëm nga skedari dhe e kthen si string (përfshirë `\n` në fund, nëse ekziston).

Programi 3 (`line_read.py`) — `'readline()'`

```python
def main():
    infile = open('philosophers.txt', 'r')

    # Lexo tre rreshta nga skedari.
    line1 = infile.readline()
    line2 = infile.readline()
    line3 = infile.readline()

    infile.close()

    # Kujdes: çdo line përmban edhe '\n'.
    print(line1) # shton edhe një '\n' nga print -> mund të duket si rresht bosh
    print(line2)
    print(line3)

if __name__ == '__main__':
    main()
```

```
John Locke

David Hume

Edmund Burke
```

Çdo thirrje e `readline()` lexon rreshtin pasues nga skedari, duke përfshirë karakterin `\n` në fund të stringut (nëse ekziston). Kjo bën që kur përdor `print(line1)`, të shfaqet një rresht bosh midis emrave, sepse `print()` shton një `\n` tjetër. Për të shmangur këtë, mund të përdorësh `print(line1, end="")`.

Kujdes me karakterin \n

Nëse rreshti i fundit në skedar **nuk** përfundon me `\n`, metoda `readline()` do të kthejë rreshtin pa karakterin e rreshtit të ri.

1.8.3 Pozicioni i leximit (read position)

Kur hap një skedar për lexim, Python mban një *pozicion leximi* (*read position*), që tregon se ku ndodhet leximi aktual në skedar. Fillimisht, ky pozicion është në fillim të skedarit. Sa herë që thërret `read()` ose `readline()`, pozicioni përparon përkatësisht në fund të skedarit ose në fillim të rreshtit tjetër.

Pas çdo thirrjeje të `readline()`, pozicioni i leximit zhvendoset në fillim të rreshtit tjetër. Kjo lejon që të lexosh skedarin rresht pas rreshti, derisa të arrish fundin.

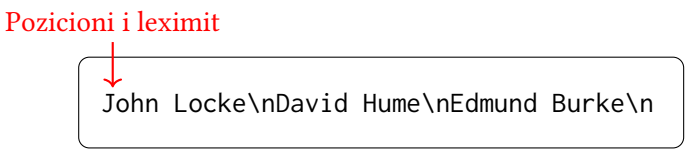

Figure 1.5: Pozicioni i leximit fillon në fillim të skedarit dhe përparon pas çdo leximi.

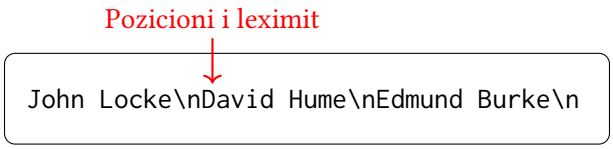

Figure 1.6: Pozicioni i leximit pas leximit të rreshtit të parë (te `David Hume`).

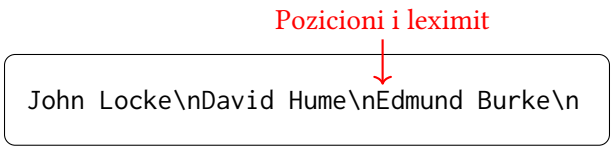

Figure 1.7: Pozicioni i leximit pas leximit të dy rreshtave (te `Edmund Burke`).

1.9 Bashkimi i \n me një *string*

Në *Programin 6-1* shkruam tre *string literals* dhe secili përfundonte me `\n`. Në praktikë, të dhënat që shkruhen në skedar vijnë zakonisht nga **variablat** (p.sh. të futura nga përdoruesi), jo nga literalet. Në këto raste, para se t'i shkruajmë në skedar, zakonisht **bashkojmë** (*concatenate*) `\n` në fund, që çdo pjesë të dhënash të jetë në **rresht më vete**.

Programi 4 (`write_names.py`)

```
def main():
    print('Shkruani emrat e tre shokëve.')
    name1 = input('Shoku #1: ')
    name2 = input('Shoku #2: ')
    name3 = input('Shoku #3: ')

    # Hape skedarin friends.txt për shkrim.
    myfile = open('friends.txt', 'w')

    # Shkruaji emrat në skedar (secilin në një rresht).
    myfile.write(name1 + '\n')
    myfile.write(name2 + '\n')
    myfile.write(name3 + '\n')

    myfile.close()
    print('Emrat u shkruan në friends.txt.')

if __name__ == '__main__':
    main()
```

```
Shkruani emrat e tre shokëve.
Shoku #1: Joe
Shoku #2: Rose
```

```
Shoku #3: Geri
Emrat u shkruan në friends.txt.
```

Pas këtij ekzekutimi, përmbajtja e `friends.txt` do të jetë (me `\n` që ndan rreshtat):

```
Joe\textbackslash nRose\textbackslash nGeri\textbackslash n
```

ose, kur hapet me një editor teksti:

```
Joe
Rose
Geri
```

Këshillë: përdorimi i *f-strings*

Rreshtat e shkrimit mund të shkruhen edhe si:

- `myfile.write(f'{name1}\n')`
- `myfile.write(f'{name2}\n')`
- `myfile.write(f'{name3}\n')`

Kjo rrit qartësinë dhe shmang `+` për bashkim *string*-ësh.

1.10 Leximi i një *string*-u dhe heqja e \n

Kur përdorim `readline()`, *string*-u i kthyer zakonisht mbaron me `\n`. Kjo është e dobishme brenda skedarit (separon elementët), por gjatë shfaqjes mund të krijojë **rreshta bosh** (sepse edhe `print()` shton një `\n`). Zgjidhja tipike është **heqja** e `\n`-së në fund me metodën `rstrip()`.

```
name = 'Joanne Manchester\n'
name = name.rstrip('\n') # -> 'Joanne Manchester'
```

Rreth 'rstrip()'

`rstrip(chars)` heq nga **ana e djathtë** çdo karakter të listuar te `chars`. Pa argument, heq hapësira të bardha (përfshirë `\n`, `\r`, `\t`).

Programi 5 (strip_newline.py)

```
def main():
    infile = open('philosophers.txt', 'r')

    line1 = infile.readline()
    line2 = infile.readline()
    line3 = infile.readline()
```

```python

    # Hiq '\n' nga fundi i secilit string (nëse ekziston).
    line1 = line1.rstrip('\n')
    line2 = line2.rstrip('\n')
    line3 = line3.rstrip('\n')

    infile.close()

    print(line1)
    print(line2)
    print(line3)

if __name__ == '__main__':
    main()
```

```
John Locke
David Hume
Edmund Burke
```

1.11 Shtimi i të dhënave në një skedar ekzistues (’a’)

Modaliteti `’w’` **zëvendëson** përmbajtjen ekzistuese (ose krijon skedarin nëse mungon). Nëse dëshironi të **shtoni** të dhëna në fund, përdorni modalitetin `’a’` (*append*):

- Nëse skedari ekziston, **nuk** fshihet; pozicioni i shkrimit vendoset në **fund**.
- Nëse skedari nuk ekziston, **krijohet** dhe pastaj shkruhet në të.

Shembull — Shtimi i të dhënave në fund të një skedari ekzistues

Supozojmë se `friends.txt` përmban:

```
Joe
Rose
Geri
```

Shtojmë emra të rinj në fund:

```python
myfile = open('friends.txt', 'a')
myfile.write('Matt\n')
myfile.close()
```

Tani `friends.txt` do të përmbajë:

```
Joe
Rose
Geri
Matt
```

Programi 6 (`file_names.py`)

```
# This program writes three lines of data
# to a file.
def main():
    # Open a file named philosophers.txt.
    outfile = open('philosophers.txt', 'a')

    # Write the names of three philosphers
    # to the file.
    outfile.write('John Locke\n')
    outfile.write('David Hume\n')
    outfile.write('Edmund Burke\n')

    # Close the file.
    outfile.close()

    print('The names were appended to philosophers.txt.')

# Call the main function.
if __name__ == '__main__':
    main()
```

```
The names were appended to philosophers.txt.
```

Kujdes: `'w'` vs `'a'`

`'w'` *zbraz* skedarin ekzistues; `'a'` *shton* në fund. Zgjidheni modalitetin sipas sjelljes që dëshironi.

1.12 Shkrimi dhe leximi i të dhënave numerike

1.12.1 Shkrimi i numrave: nga tip numerik në *string*

Metoda `write()` mund të shkruajë **drejtpërdrejt** vetëm *string*-ë në një skedar teksti. Numrat *duhet* të kthehen më parë në *string*. Për këtë përdorim funksionin e brendshëm `str()`. Nëse `num = 99`, shprehja `str(num)` kthen `'99'`.

Programi 7 (`write_numbers.py`)

```
def main():
    outfile = open('numbers.txt', 'w')

    num1 = int(input('Shkruaj një numër: '))
    num2 = int(input('Shkruaj një numër tjetër: '))
    num3 = int(input('Shkruaj edhe një numër: '))

    # Shkruaji numrat në skedar (secili në rresht më vete).
    outfile.write(str(num1) + '\n')
    outfile.write(str(num2) + '\n')
    outfile.write(str(num3) + '\n')

    outfile.close()
```

```
    print('Të dhënat u shkruan në numbers.txt')

if __name__ == '__main__':
    main()
```

```
Shkruaj një numër: 22
Shkruaj një numër tjetër: 14
Shkruaj edhe një numër: -99
Të dhënat u shkruan në numbers.txt
```

Shprehja `str(num1) + '\n'` konverton vlerën e `num1` në *string* dhe i bashkon `\n` në fund (p.sh. prodhon `'22\n'`). E njëjta gjë ndodh për `num2` dhe `num3`. Pas ekzekutimit, `numbers.txt` përmban:

```
22\n14\n-99\n
```

ose kur hapet me një editor teksti:

```
22
14
-99
```

Gabime tipike

- **Shkrim pa konvertim:** `outfile.write(num1)` *hedh gabim* sepse `write()` pret *string*.
- **Lexim pa konvertim:** vlerat e lexuara janë *string*-ë (p.sh. `'22\n'`); përpara llogaritjeve përdor `int()` ose `float()`. Përmbajtje jo-numerike do të shkaktojë `ValueError`.

1.12.2 Leximi i numrave: nga *string* në tip numerik

Kur lexon nga skedari me `readline()`, përfiton një *string*. P.sh.:

```
infile = open('numbers.txt', 'r')
value = infile.readline() # -> p.sh. '22\n'
infile.close()
```

Për të bërë llogaritje, ktheje në tip numerik:

```
infile = open('numbers.txt', 'r')
string_input = infile.readline() # '22\n'
value = int(string_input) # -> 22 (int ignoron '\n')
infile.close()
```

Shpesh e kombinojmë në një rresht:

```
infile = open('numbers.txt', 'r')
value = int(infile.readline()) # readline() -> '22\n' -> int(...) -> 22
infile.close()
```

Rreth `int()` dhe `float()`

`int()` dhe `float()` **injorojnë** `\n` në fund të *string*-ut; por do të hedhin `ValueError` nëse *string*-u nuk paraqet një numër të vlefshëm.

Programi 8 (`read_numbers.py`)

```
def main():
    infile = open('numbers.txt', 'r')

    num1 = int(infile.readline())
    num2 = int(infile.readline())
    num3 = int(infile.readline())

    infile.close()

    # Llogarit totalin.
    total = num1 + num2 + num3

    # Shfaq numrat dhe totalin.
    print(f'The numbers are: {num1}, {num2}, {num3}')
    print(f'Their total is: {total}')

if __name__ == '__main__':
    main()
```

```
The numbers are: 22, 14, -99
Their total is: -63
```

1.13 Përdorimi i cikleve për të procesuar skedarë

Shumë programe ruajnë *shuma* të mëdha të dhënash në skedarë. Në këto raste, shkrimi dhe leximi bëhen zakonisht me **cikle** (*loops*). Ciklet janë një nga konceptet më të rëndësishme në programim, sepse lejojnë që një bllok kodi të ekzekutohet shumë herë, duke përpunuar të dhëna të shumta pa përsëritur kodin manualisht. Kur punojmë me skedarë, ciklet na lejojnë të shkruajmë ose të lexojmë të gjitha të dhënat, pavarësisht sa të mëdha janë ato, në mënyrë të automatizuar dhe efikase.

Pse përdorim cikle me skedarë?

- **Automatizim:** Në vend që të shkruajmë ose lexojmë çdo vlerë me dorë, cikli e bën këtë automatikisht për çdo element.
- **Shkallëzim:** Nuk ka rëndësi nëse kemi 3 apo 3000 rreshta; kodi mbetet i njëjtë.
- **Menaxhim i lehtë i të dhënave:** Mund të përpunojmë të gjitha të dhënat në skedar, të llogarisim

statistika, të kërkojmë vlera, etj.

Në këtë pjesë do të shohim:

1) si të shkruajmë shumë vlera në një skedar brenda një `for`-loop,
2) si të lexojmë një skedar pa e ditur paraprakisht sa rreshta ka, duke zbuluar **fundin e skedarit** (EOF),
3) si ta thjeshtojmë leximin me `for` direkt mbi objektin e skedarit.

1.13.1 Shkrimi i shumë vlerave me `for` (`sales.txt`)

Kur duam të ruajmë shumë vlera në një skedar, përdorim një cikël `for` për të përsëritur procesin e marrjes së të dhënave dhe shkrimit të tyre. Kjo është shumë më efikase sesa të përsërisim kodin për çdo vlerë.

Programi 9 (`write_sales.py`)

```
def main():
    num_days = int(input('Për sa ditë ke shitje? '))

    sales_file = open('sales.txt', 'w')

    # Për çdo ditë, merre shumën dhe shkruaje në skedar.
    for count in range(1, num_days + 1):
        sales = float(input(f'Fut shitjet për ditën #{count}: '))
        sales_file.write(f'{sales}\n')

    sales_file.close()
    print('Të dhënat u shkruan në sales.txt.')

if __name__ == '__main__':
    main()
```

Në këtë shembull, cikli `for` përsëritet aq herë sa ditë ka futur përdoruesi. Çdo vlerë futet nga përdoruesi dhe shkruhet në skedar në një rresht të veçantë. Kjo mënyrë është shumë më e thjeshtë dhe më e sigurt sesa të shkruash manualisht për çdo ditë.

```
Për sa ditë ke shitje? 5
Fut shitjet për ditën #1: 1000.0
Fut shitjet për ditën #2: 2000.0
Fut shitjet për ditën #3: 3000.0
Fut shitjet për ditën #4: 4000.0
Fut shitjet për ditën #5: 5000.0
Të dhënat u shkruan në sales.txt.
```

Përmbajtja e skedarit `sales.txt`

```
1000.0
2000.0
3000.0
4000.0
5000.0
```

Sasia monetare

`float` mjafton për shembujt didaktikë. Për aplikacione financiare, konsidero `decimal.Decimal` për saktësi më të lartë në llogaritje monetare.

1.13.2 Leximi me `while` dhe zbulimi i fundit të skedarit (EOF)

Shpesh nuk e dimë *paraprakisht* sa rreshta ka një skedar. P.sh., `sales.txt` mund të ketë 5, 100 apo 1000 rreshta, varësisht çfarë futi përdoruesi. Si ta dimë kur kemi arritur **fundin e skedarit**?

Përkufizim — Çfarë është EOF?

EOF (*End Of File*) është një shenjë që tregon se nuk ka më të dhëna për t'u lexuar nga skedari. Në Python, kur thërret `readline()` dhe ke arritur fundin e skedarit, ajo kthen një **string bosh** (''). Kjo është mënyra standarde për të zbuluar fundin e skedarit në shumë gjuhë programimi.

Përkufizim — Priming read dhe cikli sentinel

Priming read është një teknikë ku lexon një vlerë para se të hysh në ciklin `while`. Kjo vlerë përdoret për të kontrolluar nëse duhet të hysh në cikël. Brenda ciklit, pas përpunimit të vlerës, lexon vlerën tjetër. Ky model quhet edhe *cikli sentinel*, sepse ndalon kur has një vlerë të veçantë (në këtë rast, stringun bosh që tregon EOF).

Ky model është shumë i përdorur në programim, jo vetëm për skedarë, por edhe për hyrje të tjera të të dhënave.

Programi 10 (`read_sales.py`)

```python
def main():
    sales_file = open('sales.txt', 'r')

    # Lexim paraprak (priming read): ende s'e kthejmë në numër.
    line = sales_file.readline()

    # Vazhdon derisa readline() kthen '' (fund i skedarit).
    while line != '':
        # Ktheje rreshtin në float.
        amount = float(line)

        # Formato dhe shfaq vlerën.
        print(f'{amount:.2f}')

        # Le rreshtin tjetër.
        line = sales_file.readline()

    sales_file.close()

if __name__ == '__main__':
    main()
```

```
1000.00
2000.00
3000.00
4000.00
5000.00
```

Në këtë shembull, cikli `while` vazhdon të lexojë dhe të përpunojë rreshta derisa të arrihet fundi i skedarit (EOF). Kjo është mënyra më klasike për të lexuar të gjitha të dhënat nga një skedar kur nuk dihet sa rreshta ka.

Mos harro konvertimin

`readline()` kthen *string*; duhet `float(...)` ose `int(...)` përpara llogaritjeve ose formatimit numerik. Tekst i pavlefshëm numerik do të shkaktojë `ValueError`.

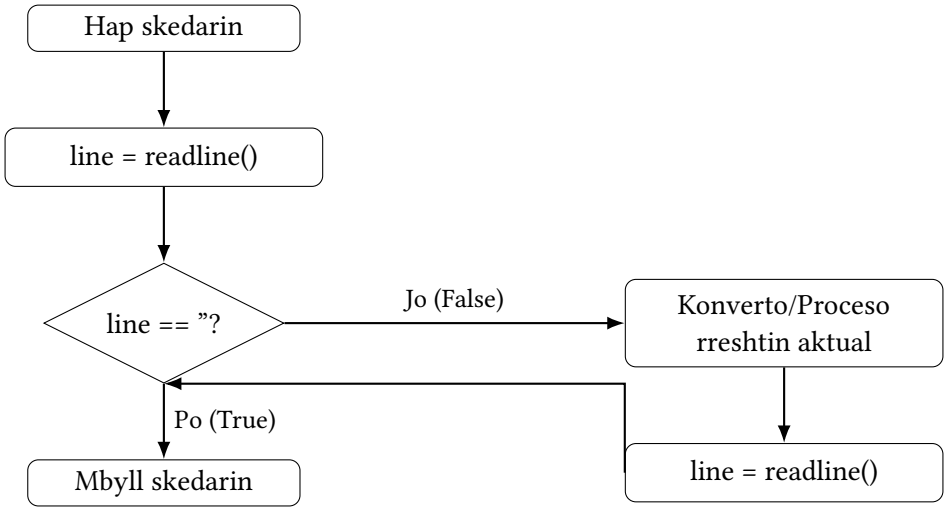

Figure 1.8: Zbulimi i fundit të skedarit me *priming read* dhe `while`.

1.13.3 Leximi më i thjeshtë: `for` direkt mbi objektin e skedarit

Në Python, mund të iterosh **drejtpërdrejt** mbi rreshtat e një skedari duke përdorur një cikël `for`. Kjo është një nga veçoritë më të fuqishme të Python-it, sepse e bën kodin më të shkurtër, më të lexueshëm dhe më të sigurt.

Përkufizim — Objekti skedar është iterable

Objekti i skedarit në Python është një *iterable*, që do të thotë se mund të përdoret në një cikël `for` për të marrë rresht pas rreshti automatikisht. Python kujdeset vetë për leximin dhe ndaljen në fund të skedarit (EOF), pa pasur nevojë për *priming read* ose kontroll manual të stringut bosh.

```python
for line in file_object:
    # përdor line (string) ...
```

Ky `for`-loop lexon rresht pas rreshti dhe **ndalon vetë** në fund të skedarit, pa pasur nevojë për *priming read* apo testim explicit për `”`.

Programi 11 (read_sales2.py)

```python
def main():
    sales_file = open('sales.txt', 'r')

    # Lexo të gjitha rreshtat një nga një.
    for line in sales_file:
        amount = float(line)
        print(f'{amount:.2f}')

    sales_file.close()

if __name__ == '__main__':
    main()
```

```
1000.00
2000.00
3000.00
4000.00
5000.00
```

Ky stil është më i thjeshtë, më i sigurt dhe më i lehtë për t'u mirëmbajtur. Python kujdeset automatikisht për ndaljen e ciklit kur arrihet fundi i skedarit.

Kur të përdorësh `for` dhe kur `while`

- Përdor `for line in file` kur do të lexosh të gjitha rreshtat një nga një, pa kushtëzime të veçanta.
- Përdor `while` me *priming read* kur ke nevojë për kontroll më të hollësishëm të rrjedhës së leximit, ose kur duhet të ndalosh në një kusht të caktuar (jo vetëm në fund të skedarit).

1.14 Përpunimi i *registreve* (records)

Përkufizim — Record dhe field

Record (*regjistër*) është një *set i plotë* të dhënash që përshkruan një njësi (p.sh. një punonjës). **Field** (*fushë*) është një copë e vetme informacioni brenda një record-i (p.sh. emri, ID-ja, departamenti). Kur shkruajmë në një skedar me akses sekuencial, fusha e një record-i shkruhen *njëra pas tjetrës* në rreshta të ndarë me `\n`.

Në programim, një **record** është një strukturë që përmbledh disa të dhëna të lidhura logjikisht, që i përkasin një entiteti të vetëm. Çdo pjesë e të dhënave quhet *field*. Për shembull, një punonjës mund të përshkruhet nga emri, ID-ja dhe departamenti. Këto janë fushat e record-it të punonjësit. Në skedarë tekst, çdo fushë zakonisht ruhet në një rresht të veçantë, dhe të gjitha fushat njëra pas tjetrës përbëjnë një record të plotë. Kjo mënyrë është shumë e përdorur për të ruajtur të dhëna të strukturuara në mënyrë të thjeshtë dhe të lexueshme.

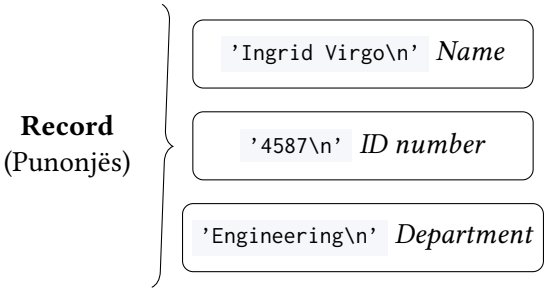

Figure 1.9: Një record me tre fusha të shkruara si rreshta në skedar.

Pse përdorim record-e në skedarë?

Record-et na lejojnë të ruajmë të dhëna të lidhura së bashku në mënyrë të organizuar. Kjo është thelbësore kur kemi shumë njësi të të njëjtit lloj (p.sh. shumë punonjës, studentë, produkte), sepse na lejon të lexojmë, përpunojmë dhe modifikojmë të dhënat për secilën njësi në mënyrë të qëndrueshme.

Figure 1.10: Disa record-e në një skedar sekuencial (fushat e çdo record-i njëra pas tjetrës).

Teori: Organizimi i të dhënave në skedarë

Kur ruajmë shumë record-e në një skedar, është shumë e rëndësishme që të gjitha record-et të kenë të njëjtën strukturë (të njëjtat fusha, në të njëjtin rend). Kjo siguron që gjatë leximit të dimë saktësisht se çfarë përfaqëson çdo rresht. Nëse struktura prishet (p.sh. mungon një fushë), të gjitha leximet pasuese do të jenë të gabuara.

1.14.1 Shkrimi i record-eve në skedar

Kur duam të ruajmë një listë të të dhënave të strukturuara (si një listë punonjësish), përdorim një cikël për të marrë të dhënat për çdo record dhe i shkruajmë në skedar, një fushë për rresht. Kjo mënyrë është shumë e përdorur në programim, sidomos për të ruajtur të dhëna që më vonë do të lexohen dhe përpunohen nga programe të tjera.

Programi 12 (`save_emp_records.py`)

```python
def main():
    # Merre numrin e record-eve.
    num_emps = int(input('Sa record-e punonjësish do të krijosh? '))

    emp_file = open('employees.txt', 'w')

    for count in range(1, num_emps + 1):
        print(f'Fut të dhënat për punonjësin #{count}')
```

```
        name = input('Name: ')
        id_num = input('ID number: ')
        dept = input('Department: ')

        # Shkruaji fushat (secila në rresht më vete).
        emp_file.write(f'{name}\n')
        emp_file.write(f'{id_num}\n')
        emp_file.write(f'{dept}\n')

        print()

    emp_file.close()
    print('Record-et u ruajtën në employees.txt.')

if __name__ == '__main__':
    main()
```

```
Sa record-e punonjësish do të krijosh? 3
Fut të dhënat për punonjësin #1
Name: Ingrid Virgo
ID number: 4587
Department: Engineering

Fut të dhënat për punonjësin #2
Name: Julia Rich
ID number: 4588
Department: Research

Fut të dhënat për punonjësin #3
Name: Greg Young
ID number: 4589
Department: Marketing

Record-et u ruajtën në employees.txt.
```

Rëndësi e renditjes së fushave

Renditja e fushave në **shkrim** duhet të përputhet saktësisht me renditjen e **leximit**. Në shembullin tonë: *name* → *id* → *dept*.

Ciklet na lejojnë të përpunojmë një numër të madh record-esh pa përsëritur kodin manualisht për secilin. Kjo është thelbësore për shkallëzim dhe mirëmbajtje të kodit. Nëse struktura e record-it ndryshon (p.sh. shtohet një fushë e re), mjafton të ndryshojmë vetëm pjesën e ciklit, jo të gjithë kodin.

1.14.2 Leximi i record-eve nga skedari

Për të lexuar record-et nga një skedar sekuencial, lexojmë *fushë pas fushe* derisa të kemi të plota të tre fushat. Përsërisim derisa të arrijmë fundin e skedarit.

Programi 13 (read_emp_records.py)

```python
def main():
    emp_file = open('employees.txt', 'r')

    name = emp_file.readline()

    while name != '':
        # Lexo fushat e tjera të record-it aktual.
        id_num = emp_file.readline()
        dept = emp_file.readline()

        name = name.rstrip('\n')
        id_num = id_num.rstrip('\n')
        dept = dept.rstrip('\n')

        # Shfaq record-in.
        print(f'Name: {name}')
        print(f'ID: {id_num}')
        print(f'Dept: {dept}')
        print()

        name = emp_file.readline()

    emp_file.close()

if __name__ == '__main__':
    main()
```

```
Name: Ingrid Virgo
ID: 4587
Dept: Engineering

Name: Julia Rich
ID: 4588
Dept: Research

Name: Greg Young
ID: 4589
Dept: Marketing
```

Gabime të zakonshme me record-e

- **Fusha të munguar/a tepruara:** Nëse harron të shkruash një fushë, *sinkronizimi* prishet dhe leximet pasuese do të jenë të gabuara.
- **Rreshta të zbrazët:** Kujdes me rreshtat bosh të padëshiruar; përdor `rstrip('\n')` gjatë leximit dhe mos shkruaj `\n` shtesë.
- **Tipet:** Të gjitha fushat lexohen si *string*. Ktheji me `int(...)` / `float(...)` vetëm kur fusha është numerike.

Leximi i record-eve nga një skedar kërkon që të dimë saktësisht sa fusha ka çdo record dhe në çfarë rendi janë shkruar. Nëse struktura është e qëndrueshme, mund të lexojmë të dhënat në mënyrë të sigurt dhe të përpunojmë çdo record

veçmas. Kjo teknikë është bazë për shumë aplikacione që ruajnë të dhëna të strukturuara në skedarë tekst.

1.15 Përdorimi i `with` me skedarët (context manager)

Në Python, është thelbësore që skedarët të mbyllen gjithmonë pasi të përfundojë puna me ta. Nëse harron të thërrasësh `close()`, mund të humbasësh të dhëna, të bllokosh skedarin për përdorime të tjera, ose të shkaktosh gabime të paparashikuara në program. Për ta bërë këtë proces më të sigurt dhe më të thjeshtë, Python ofron fjalën kyçe `with`, e cila përdor një mekanizëm të quajtur *context manager*.

1.15.1 Çfarë është një context manager?

Një **context manager** është një objekt që përcakton se si të menaxhohet një burim (si një skedar, një lidhje me bazë të dhënash, etj.) gjatë një blloku të caktuar të kodit. Ai siguron që burimi të hapet në fillim të bllokut dhe të mbyllet automatikisht në fund, edhe nëse ndodh një gabim (*exception*) gjatë ekzekutimit të kodit brenda bllokut.

Teori: Pse është i rëndësishëm context manager-i?

- **Menaxhim automatik i burimeve:** Skedarët, lidhjet me bazë të dhënash, rrjetet, etj., janë burime që duhet të mbyllen kur nuk nevojiten më. Context manager-i siguron që ato të mbyllen gjithmonë, pa pasur nevojë për kod shtesë.
- **Mbrojtje ndaj gabimeve:** Edhe nëse ndodh një *exception* brenda bllokut, context manager-i kujdeset që burimi të mbyllet siç duhet.
- **Kod më i pastër dhe më i lexueshëm:** Sintaksa e `with` bën që kodet për hapje/mbyllje të burimeve të jenë më të qarta dhe më të lehta për t'u mirëmbajtur.

Sintaksa bazë është:

```
with open('emri_skedarit.txt', 'w') as file:
    # Këtu shkruan ose lexon nga skedari

    file.write('Përshëndetje!')
# Kur del nga blloku, skedari mbyllet automatikisht
```

Kur përdor `with`, Python thërret automatikisht `file.close()` sapo del nga blloku, edhe nëse ndodh një gabim brenda tij. Kjo e bën kodin më të sigurt dhe shmang rrezikun e lënies së skedarëve të hapur.

1.15.2 Shembuj të përdorimit të `with`

Programi 14 (`with_write.py`)

```
def main():
  with open('philosophers.txt', 'w', encoding='utf-8') as f:
      f.write('John Locke\n')
      f.write('David Hume\n')
      f.write('Edmund Burke\n')

if __name__ == '__main__':
    main()
```

Përmbajtja e skedarit `philosophers.txt`

```
John Locke
David Hume
Edmund Burke
```

Në këtë shembull, skedari `philosophers.txt` hapet për shkrim dhe mbyllet automatikisht sapo del nga blloku `with`. Nuk ka nevojë të thërrasësh `f.close()`.

Programi 15 (`with_read.py`)

```python
def main():
    with open('sales.txt', 'r', encoding='utf-8') as f:
        for line in f:
            amount = float(line)
            print(f'{amount:.2f}')

if __name__ == '__main__':
    main()
```

```
1000.00
2000.00
3000.00
4000.00
5000.00
```

Këtu, çdo rresht i skedarit lexohet dhe konvertohet në numër. Edhe nëse ndodh një gabim gjatë leximit, skedari do të mbyllet automatikisht.

Programi 16 (`with_multiple.py`)

```python
def replace_department(emp_id, new_dept):
    with open('employees.txt', 'r', encoding='utf-8') as src, \
         open('employees.tmp', 'w', encoding='utf-8') as dst:

        while True:
            name = src.readline()
            if name == '':
                break
            id_num = src.readline()
            dept = src.readline()

            name = name.rstrip('\n')
            id_num = id_num.rstrip('\n')
            dept = dept.rstrip('\n')

            if id_num == emp_id:
                dept = new_dept

            dst.write(f'{name}\n{id_num}\n{dept}\n')

def main():
```

```
    replace_department('4587', 'HR')
    replace_department('4588', 'Finance')
    replace_department('4789', 'IT')

if __name__ == '__main__':
    main()
```

Përmbajtja e skedarit `employees.txt` pas ekzekutimit të `with_multiple.py`

```
Ingrid Virgo
4587
HR
Julia Rich
4588
Finance
Greg Young
4589
Marketing
```

Ky shembull tregon si mund të lexosh nga një skedar dhe të shkruash në një tjetër në të njëjtën kohë, duke përdorur dy `with` të bashkuara. Të dy skedarët do të mbyllen automatikisht, edhe nëse ndodh një gabim gjatë përpunimit.

1.15.3 Çfarë ndodh jashtë bllokut `with`?

Pasi del nga blloku `with`, objekti i skedarit është mbyllur dhe nuk mund të përdoret më. Nëse tenton të lexosh ose shkruash pas kësaj pike, do të marrësh gabim. Kjo sjellje është e qëllimshme për të siguruar që burimet të mos përdoren gabimisht pasi janë mbyllur.

Kujdes: përdorimi jashtë bllokut `with`

Çdo përpjekje për të përdorur objektin e skedarit pasi ka përfunduar blloku `with` do të shkaktojë gabim të tipit `ValueError: I/O operation on closed file`.

Përfitimet kryesore të `with`

- Siguron mbyllje automatike të skedarëve, edhe në rast gabimi.
- E bën kodin më të pastër, më të shkurtër dhe më të lehtë për t'u mirëmbajtur.
- Është praktikë e rekomanduar në Python për çdo operacion me skedarë ose burime të tjera të jashtme.

Teori: Context manager në përdorime të tjera

`with` nuk përdoret vetëm për skedarë. Mund të përdoret për menaxhimin e lidhjeve me bazë të dhënash, bllokimin e thread-eve, menaxhimin e burimeve të rrjetit, etj. Çdo objekt që implementon metodat speciale `__enter__()` dhe `__exit__()` mund të përdoret si context manager.

Përmbledhje e kapitullit

- Kuptuam nevojën për skedarë dhe hapat *hap—proceso—mbyll*.
- Dalluam llojet e skedarëve (tekst/binar) dhe aksesin (sekuencial/direkt).
- Përdorëm `open()` me `'r'`, `'w'`, `'a'` dhe rrugë relative/absolute (`raw string` në Windows, `pathlib` për portabilitet).
- Shkruam/lexuam tekst; menaxhuam `\n` dhe `rstrip()`.
- Konvertuam numra në/nga string (`str()`, `int()`, `float()`).
- Lexuam me `while` (EOF) dhe me `for` direkt mbi objektin e skedarit.
- Organizuan të dhënat si **record-e** me **field-e** dhe i ruajtëm/lexuam në rend të qëndrueshëm.
- Përdorëm **`with`** për mbyllje automatike dhe kod më të sigurt, për një ose më shumë skedarë.